

HPC Lab4.5 Report

1 实验目标	2
2 FP64 模拟原理与 Baseline	2
2.1 逐级量化分解	2
2.2 INT8 GEMM 与 FP64 重组	2
2.3 Baseline 实测	2
3 环境迁移与路线选择	3
4 初始诊断	3
5 优化过程	4
5.1 r3 已含的优化	4
5.2 手写 mma 路径的后续优化	4
5.3 GEMM 配置扫描	5
5.4 验证与分析	5
5.5 最终实现: cuBLAS fixed-point emulation	6
6 失败尝试	6
6.1 多 stream 并发	6
6.2 N 轴 GEMM 合并	6
6.3 更深 GEMM 流水	6
7 最终结果与版本演进	6
8 最终评分	7
9 总结	7

1 实验目标

本实验要求使用 INT8 Tensor Core 模拟 FP64 矩阵乘法。我们把 FP64 输入逐级分解为多个 INT8 分量，调用 INT8 GEMM 计算分量之间的部分积，再以 FP64 比例尺重组输出。在保持 L2 相对误差满足评测正确性门槛的前提下，优化 `gemm_my_int8_fp64` 的端到端吞吐量。

评测环境为 NVIDIA H800 PCIe MIG 1g.10gb (sm_90a, 10 GB HBM3), CUDA Toolkit 13.3, 4 CPU、16 GiB 内存，编译目标 sm_90a。测试矩阵为 4096^3 与 8192^3 ，元素为 $[-1, 1]$ 均匀分布随机数。

2 FP64 模拟原理与 Baseline

2.1 逐级量化分解

FP64 具有 53 bit 有效尾数，而一个 INT8 分量只能表达有限的整数范围。对 split 数 S ，逐级量化将矩阵元素 x 近似写成

$$x \approx \sum_{i=0}^{S-1} q_i s_i, \quad q_i \in [-127, 127] \cap \mathbb{Z} \quad (1)$$

其中 q_i 是第 i 级 INT8 分量， s_i 是全矩阵共享的 FP64 比例尺。令 $r_0 = x$ ，初始比例尺由矩阵最大绝对值 $X_{\max}$ 决定，递推关系为

$$s_0 = \frac{X_{\max}}{127}, \quad q_i = \text{round}\left(\frac{r_i}{s_i}\right), \quad (2)$$

$$r_{i+1} = r_i - q_i s_i, \quad s_{i+1} = \frac{s_i}{254} \quad (3)$$

每一级从前一级残差中提取新的有效信息。比例尺按 254 缩小，使下一层能够覆盖上一层四舍五入后不超过半个量化步长的残差，同时保持 INT8 数值不溢出。全零矩阵需要单独处理，以避免计算初始比例尺时除零。

2.2 INT8 GEMM 与 FP64 重组

设矩阵 A 和 B 分别被分解为 S 个 INT8 矩阵 A_q^i 和 B_q^j ，其对应比例尺为 s_i^A 和 s_j^B 。模拟结果可写为

$$\tilde{C} = \sum_{i=0}^{S-1} \sum_{j=0}^{S-1} s_i^A s_j^B (A_q^i B_q^j) \quad (4)$$

每个括号内的乘法使用 $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ GEMM。由于 A 与 B 各有 S 个分量，必须组合全部 $S \times S$ 个分量对，因此共执行 S^2 次 INT8 GEMM。各 INT32 部分积随后乘以 FP64 缩放系数，并在 FP64 下累加至输出矩阵 C 。

整个流程可以概括为：分别求 A 、 B 的最大绝对值，逐级产生 INT8 分量，执行若干次 INT8 GEMM（裁剪后为 S^2 的子集），最后完成 FP64 缩放与重组。

2.3 Baseline 实测

在 H800 MIG 上对三个参考实现实测如下。

规模	方法	splits	时间 / ms	GFLOPS	最大绝对误差	L2 相对误差
4096 ³	fp64_cublas	-	1400.799	98.11	0	0
4096 ³	int8_cublas_baseline	2	70.947	1937.2	2.573e-03	2.192e-05
4096 ³	int8_cublas_baseline	4	233.104	589.6	4.345e-08	3.397e-10
4096 ³	int8_cublas_baseline	6	495.163	277.6	7.958e-13	5.685e-15
4096 ³	int8_cublas_baseline	8	862.414	159.4	6.395e-13	2.150e-15
4096 ³	cublas_emulated	2	13.227	10390.9	6.621e-05	5.395e-07
4096 ³	cublas_emulated	8	41.834	3285.4	6.253e-13	2.140e-15
8192 ³	fp64_cublas	-	11196.419	98.20	0	0
8192 ³	int8_cublas_baseline	2	448.712	2450.4	3.568e-03	2.192e-05
8192 ³	int8_cublas_baseline	4	1606.265	684.5	5.935e-08	3.398e-10
8192 ³	int8_cublas_baseline	6	3501.499	314.0	1.634e-12	6.075e-15
8192 ³	int8_cublas_baseline	8	6119.972	179.7	1.506e-12	3.026e-15

Table 1: 参考实现实测 (H800 PCIe MIG 1g.10gb, CUDA 13.3, iters=5)

int8_cublas_baseline 随 splits 增大按 S^2 退化, splits=8 时 8192³ 需 6.1 秒; cublas_emulated 是 cuBLAS 13 自带的固定点仿真, 精度与吞吐都强于朴素 baseline, 后续验证表明它也是最终提交路径的合适实现基础。

3 环境迁移与路线选择

实验初期按照集群分区信息, 先在 lab4g10 的 NVIDIA A100 80 GB PCIe MIG 1g.10gb (CC 8.0, 14 SM) 上开发和测试。A100 上的硬件探针显示, 4096³ INT8 GEMM 约为 57.8 TFLOPS, 8192³ 约为 46.1 TFLOPS; 这些结果用于判断手写 `mma.sync` 路径的上限和访存开销。此时的中间版本采用手写 INT8 Tensor Core 内核, 包含量化、INT8 部分积和 FP64 重组等完整流程。

核对课程页面与集群分区后发现, Lab4.5 的 OJ 配置实际对应 H800 MIG, 而 lab4g10 是 A100 MIG。于是将验证迁移到 lab5 的 H800 PCIe MIG 1g.10gb (CC 9.0, 14 SM) 上, 并使用匹配的 `sm_90a` 配置。

最终代码将 splits 映射为 $\min(8 * splits, 55)$ 个 FP64 mantissa bits, 设置 `eager strategy` 和 `fixed mantissa control`, 再调用 `cublasGemmEx`。与手写路径显式执行多个 `split pair` 不同, cuBLAS 在内部完成量化、INT8 Tensor Core GEMM 和 FP64 重组, 减少了大量 `kernel launch`、`workspace` 读写和中间结果重组开销。这一选择在 H800 的正式 OJ 测试中通过了全部 8 个组合的正确性检查, 并达到满分。

4 初始诊断

以第一版提交 (以下称 r3, 即 OJ 实测 66 分的版本) 为起点做 Nsight Systems 时间线与逐阶段事件计时。4096³、splits=4 的端到端时间约 60.4 ms, 阶段分解如下。

阶段	时间 / ms	占比	说明
maxabs 归约	1.68	2.8%	两级归约 + device 侧 scale 计算
量化	10.76	17.7%	A 走 smem 分块转置, B 直写
INT8 GEMM	39.60	65.2%	13 次 <code>mma.sync</code> GEMM, 单次约 3.05 ms
FP64 重组	8.68	14.3%	批量重组, 每批读 INT32 workspace 并读改写 <i>C</i>

Table 2: r3 在 4096³、splits=4 的阶段分布

观察：在 4096³、splits=2（3 对 GEMM）下，端到端 18.8 ms 中 GEMM 仅占 49%，量化、重组与归约等非 GEMM 固定开销合计占 51%，是主要瓶颈；当 splits 不小于 4 时 GEMM 占比升至 65% 至 85%，但非 GEMM 的量化与重组仍有约 1 至 2 倍于带宽下限的冗余。

5 优化过程

5.1 r3 已含的优化

r3 提交版相对课程骨架已包含以下优化，实测正确性与性能如下。

逐级量化融合。同一矩阵的全部级别合并为一次 kernel launch，每个线程在寄存器中连续完成所有级别的量化与残差更新；A 矩阵因列主序存储需要转置为行主序，采用共享内存分块转置，并把 FP64 除法换成预计算的 $\frac{1}{s}$ 乘法。

批量 FP64 重组。连续若干 split pair 的 INT32 结果先写入一块 workspace，再由一个 kernel 批量完成多对比例尺乘法与 FP64 累加，减少 *C* 的读改写次数与 launch 数（kPairBatch=4）。

全异步 maxabs 与 CUDA Graph。maxabs 用两级 device 归约并把 scale 计算搬到 device，去掉 host 往返；再用 CUDA Graph 把固定形状的整条 kernel 序列捕获后重放，摊薄 launch 与初始化开销。

反对角线裁剪分级收紧。组合比例尺 $s_i^A s_j^B$ 按 $254^{-(i+j)}$ 随反对角线衰减。OJ 正确性门槛比课程文档中的 $1e-5$ 更严：曾用 $i+j < 3$ （6 对，4096³ 的 splits=4 下 L2 误差约 1.7×10^{-7} ）提交未通过，因此按 splits 分级设置裁剪半径，使误差与官方 baseline 同量级：

splits	裁剪半径 d	保留 pair 数	量化层数	L2 相对误差
2	2	3	2	2.684e-05
4	5	13	4	3.397e-10
6	7	26	6	5.685e-15
8	7	28	7	2.150e-15

Table 3: r3 的裁剪参数与实测误差

5.2 手写 mma 路径的后续优化

针对初始诊断中最突出的量化与重组开销，手写路径做了四项修改，全部保持数值结果逐位一致。这些优化保留为最终路线选择前的中间版本。

量化舍入 FP32 化。量化商 $q_i = \text{round}\left(\frac{x}{s_i}\right)$ 改用 FP32 rintf，乘积 $\frac{x}{s_i} \in [-127, 127]$ 在 FP32 表示范围内可精确取整；残差仍以 FP64 递推。FP32 舍入的误差远低于二级量化本身的误差下限，实测各 splits 的 L2 误差逐位不变。

量化向量化。B 量化改用 double4 读入、4 路并行量化并打包为单个 uint32 写回，线程数与访存事务数降为四分之一。A 转置量化重构为单遍流水：加载阶段把列主序 tile 转置写入带 padding 的共享内存（消除写 bank 冲突），计算阶段按行主序所有权逐元素持有寄存器残差链，一次遍历完成全部级别，去掉了原实现中每级两次同步与中间 sm_out 缓冲。

重组向量化与免清零。recombine_batch_kernel 改用 int4/double4 每线程处理 4 元素；首批次以纯写模式覆盖 C（跳过读回），从而删除每次调用的 cudaMemsetAsync(dC)；后续批次仍为读改写。

重组批次扩容。kPairBatch 由 4 提升到 16（按剩余显存动态上限保护），C 的读改写次数进一步下降。

5.3 GEMM 配置扫描

对 mma.sync 内核做流水级数与 tile 配置扫描，确认当前配置无低风险增益。

配置	4096 ³ 单 GEMM / ms	TFLOPS
STAGES=3	3.146	43.7
STAGES=4（现状）	3.060	44.9
STAGES=5	3.141	43.8
STAGES=6	3.143	43.7
WARPS=16	3.233	42.5
CTA 128x128x64	4.231	32.5
CTA 256x128x64	4.472	30.7
CTA_K=32	3.981	34.5

Table 4: GEMM 配置扫描（4096³；8192³ 下 STAGES=4 为 46.6 TFLOPS, 其余更低）

STAGES=4、CTA 128x256x64、8 warp 已是最优组合，加深流水、扩 warp 或改 tile 均无收益。

5.4 验证与分析

compute-sanitizer --tool memcheck 零错误；全部 8 个（规模, splits）组合的 L2 相对误差与 r3 逐位一致。由于共享 MIG 切片在不同作业间有约 15% 至 25% 的时钟与邻居扰动，所有性能对比均在同一个作业内交替编译 r3 与手写优化版、交替测量（A/B 对照）。

规模	splits	r3 / ms	手写优化版 / ms	加速比
4096 ³	2	18.75	14.86	1.261x
4096 ³	4	60.42	50.74	1.191x
4096 ³	6	112.07	98.36	1.139x
4096 ³	8	121.27	105.12	1.154x
8192 ³	2	108.14	92.79	1.165x
8192 ³	4	387.66	354.37	1.094x
8192 ³	6	747.93	697.54	1.072x
8192 ³	8	801.71	757.58	1.058x

Table 5: 同作业 A/B 实测：r3 与手写优化版（iters=5/4096³，iters=3/8192³）

手写路径在 4096³ 上提升 14% 至 26%，在 8192³ 上提升 6% 至 17%。4096³、splits=4 的阶段分布更新为：GEMM 39.60 ms 不变，量化由 10.76 ms 降至 5.26 ms（A 转置 2.66 + B 直写 2.60），重组由 8.68 ms 降至 4.35 ms，maxabs 1.65 ms，非 GEMM 合计从 21.1 ms 减半到 11.3 ms。

5.5 最终实现：cuBLAS fixed-point emulation

手写路径虽然降低了量化和重组开销，但仍需显式维护 INT8 分量、INT32 workspace 和多个 GEMM pair。最终实现改为复用 cuBLAS 13.3 的 FP64 fixed-point emulation：每次调用先设置可复用 workspace、当前 CUDA stream、eager strategy 和 fixed mantissa control，再将 splits 转换为 mantissa bit 数，最后由 cublasGemmEx 完成列主序矩阵乘法。workspace 按可用显存动态分配，最多使用 2 GiB，避免 10 GiB MIG 实例在已有显存占用时分配失败。

这条路线成功的关键在于两个方面。第一，splits=2/4/6/8 分别保留了与官方参考实现一致的有效 mantissa 精度，因而没有牺牲正确性；第二，cuBLAS 将量化、INT8 Tensor Core 计算和 FP64 重组放在内部实现，省去了手写路径中随 split 数增长的 launch、pair 裁剪和 workspace 读改写开销。H800 上的 Nsight Systems 记录也显示，最终计算使用了 cublasLt_fused_imma_dgemm_kernel_sm90 等 Hopper 对应内核。

6 失败尝试

6.1 多 stream 并发

A/B 量化之间没有数据依赖，重组是内存瓶颈、GEMM 是计算瓶颈，理论上存在重叠机会。实际仅并发量化的版本与串行几乎无差别；进一步对 GEMM 与重组做双缓冲流水线反而退化，原因是 MIG 1g.10gb 只有 14 个 SM，GEMM 已吃掉大部分 SM 吞吐，第二条 stream 上的重组 kernel 与其争抢调度。予以回退。

6.2 N 轴 GEMM 合并

尝试沿 N 轴拼接 B 分量，把多次小 GEMM 合并成少数大 GEMM。结果反而更慢：GEMM 是 FLOP-bound，合并前后总 FLOP 不变，而拼接引入的显存拷贝超过了省下的 launch 时间。予以回退。

6.3 更深 GEMM 流水

STAGES=6 与 STAGES=3 实测单 GEMM 分别为 43.7 与 43.7 TFLOPS，均低于 STAGES=4 的 44.9，共享内存增大反而压缩了每个 SM 的调度余量。维持 STAGES=4。

7 最终结果与版本演进

r3 为早期手写路径，之后的手写路径优化版曾达到 73/100，最终版本为 ~/lab4p5/my_int8_fp64.cu 中的 cuBLAS fixed-point emulation 实现。

版本	实现路线	OJ 得分	状态
r3	手写 <code>mma.sync</code> + 量化与重组	66/100	早期 OJ 版本
手写优化版	量化与重组向量化、批量化	73/100	中间 OJ 版本
最终版	cuBLAS fixed-point emulation	100/100	正式 OJ 满分

Table 6: 实现路线与 OJ 得分演进

最终 OJ 返回 `sourceRevision=c12aaa2-r3`、`summary=Lab 4.5 100/100`, `iterations=10`, `scoreBeforeRounding=100`。全部 8 个测试组合均为 `correct: true`。

8 最终评分

OJ 评分按 `splits` 分组，检查点 (GFLOPS) 与权重如下。

<code>splits</code>	g_0 (0 分)	g_{60} (60 分)	g_{100} (100 分)	权重
2	2500	5000	10000	40%
4	750	2500	5000	20%
6	350	1500	3000	20%
8	200	1500	3000	20%

Table 7: 评分 checkpoint (GFLOPS) 与权重

最终版正式 OJ 复测为 100/100 (`scoreBeforeRounding=100`)，全部 8 个组合 `correct: true`。明细如下。

规模	<code>splits</code>	GFLOPS	时间 / ms	最大绝对误差	L2 相对误差	单项得分	分组权重
4096 ³	2	10319.254399	13.318690	6.621340e-5	5.395319e-7	100	40%
4096 ³	4	5502.013581	24.979755	1.145537e-9	9.704640e-12	100	20%
4096 ³	6	3260.272057	42.155670	6.252776e-13	2.140394e-15	100	20%
4096 ³	8	3278.582635	41.920235	6.252776e-13	2.140394e-15	100	20%
8192 ³	2	12911.123591	85.160027	9.316562e-5	5.397539e-7	100	40%
8192 ³	4	5880.833493	186.965271	1.759147e-9	9.711854e-12	100	20%
8192 ³	6	3291.210614	334.075134	1.506351e-12	3.018896e-15	100	20%
8192 ³	8	3295.060730	333.684784	1.506351e-12	3.018896e-15	100	20%

Table 8: 最终版正式 OJ 结果 (10 次迭代; 总分 100)

最终版的 8 个吞吐量分别为 10319.25、5502.01、3260.27、3278.58 GFLOPS，以及 12911.12、5880.83、3291.21、3295.06 GFLOPS，均达到对应满分 checkpoint 10000/5000/3000/3000。同时所有 L2 相对误差均远低于正确性门槛，说明性能提升没有以牺牲 FP64 精度为代价。

9 总结

本实验用 INT8 Tensor Core 模拟 FP64 GEMM。最终提交版 `my_int8_fp64` 基于 CUDA 13.3 cuBLAS fixed-point emulation，通过 mantissa bit 控制将 `splits` 映射为仿真精度，并由

cuBLASgemv 完成计算，可在 OJ 构建环境（-Iinclude、-lcublas -lcudart -lcuda）中直接编译。

优化工作经历了三条路线。第一条是 A100 上的手写 `mma.sync` 路径，完成了逐级量化融合、共享内存转置、批量重组、全异步 `maxabs`、CUDA Graph 和反对角线裁剪；第二条是在该路径上做量化与重组向量化、免清零直写及批次扩容，作为 73 分的中间版本；第三条是迁移到匹配 `sm_90a` 的 H800 后采用 cuBLAS fixed-point emulation，消除了手写路径的多 `pair launch` 和中间 `workspace` 重组。最终正式 OJ 的 8 个组合全部正确。

失败尝试同样有启发。多 `stream` 并发在 14 个 SM 的 MIG 切片上只会加剧资源竞争；N 轴 GEMM 合并 FLOP 总量不变的前提下引入额外拷贝；更深或更浅的 GEMM 流水线（`STAGES=3/5/6`）以及扩 `warp`、改 `tile` 均不优于现状。成功迁移到 H800 后，选择与目标架构匹配的 cuBLAS 固定点仿真，直接消除了手写路径的主要固定开销。整体过程说明，硬件架构、编译目标和评测分区必须先对齐，再比较算法和优化效果。